

Você sabe o que é **BEHAVIOR** na AWS?

```
terminal - cloudfront-architecture

# Como desacoplar containers Docker e economizar CPU:

$ aws cloudfront create-distribution \
  --origins Id=S3-Assets,Id=ALB-Backend \
  --default-cache-behavior TargetOriginId=S3-Assets

-> Roteamento na Edge com multiplas origens e zero desperdicio.
```

Aprenda como dividir tráfego de imagens e APIs dinâmicas
usando um único domínio, acelerando o carregamento da sua aplicação.

Não queime CPU cara com arquivos estáticos!

? No Container Docker

- Imagem incha de 80MB para 1.5GB+
- Servidor gasta CPU/RAM caras apenas lendo arquivos do disco.
- Auto Scaling lento na Black Friday.
- Uploads de usuários somem se reiniciar.

? No S3 + CloudFront

- Imagem Docker limpa e rápida (<100MB).
- Entrega na Edge em milissegundos.
- 99.999999999% de durabilidade.
- Custo de centavos por GB armazenado.
- Container 100% focado na API.

? REGRA DE OURO DA ARQUITETURA MODERNA

Container processa regra de negócio (CPU & Memória).

Amazon S3 + CloudFront entregam e armazenam arquivos estáticos.

O que é o CloudFront BEHAVIOR?

É a regra de trânsito na Edge que inspeciona a URL de cada requisição.

1. Origem de Destino (Routing)

Define para qual backend a rota será enviada:

- **Bucket Amazon S3** (para fotos, banners e front-end SPA)
- **Application Load Balancer / API Gateway** (para microsserviços)

2. Política de Cache e Headers (Cache Policy)

Define se a resposta fica em cache e por quanto tempo:

- **Imagens & Assets: Cache agressivo** (TTL de 1 ano, compressão Brotli).
- **Checkout & APIs: Cache ZERO** (TTL=0, repassa Cookies e Auth).

Roteamento Inteligente em 1 Único Domínio (mystore.com)

PATH PATTERN

/images/* & /static/*

DESTINO: Bucket S3

Cache longo (1 ano) + Gzip/Brotli. Alívio total para o backend.

PATH PATTERN

/api/checkout/* & /api/cart/*

DESTINO: Container (ALB)

Cache ZERO. Repassa Headers, Cookies e métodos POST/PUT/DELETE.

PATH PATTERN

Default (*)

DESTINO: Bucket S3

Entrega o index.html da Single Page App (React, Vue, Angular).

Cuidado com a Ordem de **PRECEDÊNCIA!**

AVALIAÇÃO DE CIMA PARA BAIXO (Top to Bottom) ??

O CloudFront para na PRIMEIRA regra correspondente.

- Precedência 0: `/images/` -> Avaliado primeiro
- Precedência 1: `/api/` -> Avaliado em seguida
- Precedência 2: `Default (*)` -> Fallback final

Se colocar o coringa (*) no topo, ele anula todas as outras regras!

? SALVE ESTE POST!

Deixe guardado para consultar quando for desenhar ou revisar sua próxima arquitetura de microsserviços na AWS.